

Name: Leigh Thurgood
Date: June 3, 2026
Course: Foundations Of Databases & SQL Programming (FDN130)

Assignment 7 – Functions (Module 7)

Introduction

A user defined function is just a piece of SQL you write once, give a name, and then call whenever you need it. Instead of copying the same logic into every query, you wrap it up and reuse it. This paper looks at when it makes sense to reach for a UDF, and how the three types differ once you do.

Explain when you would use a SQL View.

The short version is: use one when you find yourself writing the same logic over and over. If there's a calculation or a lookup that keeps showing up across your queries, pulling it into a function means you write it once and fix it in one place if it's ever wrong. It also makes the queries that call it a lot easier to read, since a clean function name tells you what's happening without making you decode the math every time. The other big reason is consistency. If everyone on a project calls the same function to figure out something like a KPI or a formatted value, you know they're all getting the same answer instead of each person rolling their own slightly different version. The thing to watch out for is performance. Functions are handy, but if you stack a scalar function inside a query that runs across a huge table, it can slow things down, so they're best when the convenience is worth it.

Explain the differences and similarities between a View, Function, and Stored Procedure.

Three kinds, and they differ mostly in what they hand back. A scalar function returns one single value, like a formatted price or a yes/no flag. You give it some input and it gives you back one thing. An inline table-valued function returns a whole table, but the catch is it has to be written as a single SELECT statement. It's basically a parameterized view, and because it's just one query under the hood, the engine handles it efficiently. A multi-statement table-valued function also returns a table, but it lets you build that table up over several steps. You declare a table variable, run whatever logic you need to fill it, and then hand it back. That extra flexibility is nice when one SELECT won't cut it, but it usually comes at a cost in speed compared to the inline kind.

Conclusion

Pick the simplest one that does what you need. Scalar handles single values, inline handles tables you can write in one SELECT, and multi-statement is there for when neither of those will fit. Save the harder option for when you have to use it.